

HAZELCAST

Deep Dive: Theory to Production

with Spring Boot & Payment Domain Examples

Topics Covered

Core Architecture • Data Structures •
Configuration
Spring Boot Integration • Caching • Sessions
Distributed Computing • Best Practices

Table of Contents

Chapter 1: Introduction to Hazelcast	4
1.1 What is Hazelcast?	4
1.2 Problems Hazelcast Solves	4
1.3 Hazelcast Editions	5
1.4 Hazelcast vs Alternatives	5
Chapter 2: Hazelcast Architecture — How It Actually Works	6
2.1 The Cluster	6
What is a Node vs a Member vs a Client?	6
2.2 Member Discovery — How Nodes Find Each Other	6
2.3 Data Partitioning — How Data Is Distributed	7
2.4 Backup & High Availability	7
2.5 Split-Brain Protection	8
2.6 Embedded Mode vs Client-Server Mode	8
Chapter 3: Setup — Spring Boot + Hazelcast from Zero	9
3.1 Maven Dependencies	9
3.2 Hazelcast Configuration — Java Config	10
3.3 YAML Configuration (Alternative)	11
3.4 application.properties Settings	12
3.5 Enable Spring Cache in Main Class	12
3.6 Server Mode — Running Standalone Hazelcast	12
3.7 Boot as Hazelcast Client (ClientConfig)	13
3.8 Docker Compose — Full Local Cluster Setup	13
Chapter 4: IMap — The Core Data Structure	13
4.1 What is IMap?	13
4.2 Getting an IMap Instance	13
4.3 Core Methods — Complete Reference	14
Basic CRUD Operations	14
Async Operations (Non-Blocking)	15
putAll — Batch Write (High Performance)	15
Locking Methods	16
Entry Processors — Server-Side Execution	17
Predicates — Server-Side Filtering	18
Indexes — Speed Up Predicate Queries	19
Map Event Listeners	20
Chapter 5: Other Distributed Data Structures	21
5.1 IQueue — Distributed Queue	21
5.2 ITopic — Distributed Pub/Sub	22
5.3 ISet — Distributed Set	24

5.4 IList — Distributed List	24
5.5 IAtomicLong — Distributed Counter	25
5.6 FencedLock — Distributed Lock (CP Subsystem)	26
Chapter 6: Spring Cache Integration	27
6.1 How Spring Cache Works with Hazelcast	27
6.2 @Cacheable — Cache Method Results	27
6.3 @CachePut — Always Execute and Update Cache	28
6.4 @CacheEvict — Remove from Cache	29
Chapter 7: Spring Session with Hazelcast	30
7.1 The Problem Spring Session Solves	30
7.2 Dependency & Configuration	30
Chapter 8: Serialization — How Data Travels Between Nodes	32
8.1 Why Serialization Matters	32
8.2 Implementing IdentifiedDataSerializable	32
Chapter 9: Advanced Patterns — Payment Domain	34
9.1 Idempotency Key Pattern	34
9.2 Near Cache — Reducing Network Hops	35
9.3 MapStore — Write-Through / Write-Behind Persistence	35
9.4 Distributed Transaction Pattern	37
Chapter 10: Management, Monitoring & Best Practices	38
10.1 Hazelcast Management Center	38
10.2 Metrics via Spring Boot Actuator + Micrometer	38
10.3 Best Practices Summary	39
10.4 Common Pitfalls	40
Chapter 11: Quick Reference Card	41
11.1 Data Structure Selection Guide	41
11.2 Configuration Cheat Sheet	41

Chapter 1: Introduction to Hazelcast

1.1 What is Hazelcast?

Hazelcast is an open-source, distributed, in-memory data grid (IMDG) and stream-processing platform written in Java. It lets multiple JVM processes — running on different machines — share data structures in memory as if they were local, while automatically handling replication, partitioning, and failover behind the scenes.

In simpler terms: imagine you have a payment service running on 5 servers. Without Hazelcast, each server has its own memory. A payment session created on Server 1 is invisible to Server 3. With Hazelcast, all 5 servers share one logical memory space. Any server can read or update any data instantly.

 Real-world analogy

Think of Hazelcast like a shared whiteboard in an office. Every employee (server) can walk up to it, read what's there, and write new information. Without it, each employee has a personal notepad — they never see each other's notes until a meeting (database sync), which takes time.

1.2 Problems Hazelcast Solves

Problem	Without Hazelcast
Data Silos	Multiple app instances each cache data locally → stale or inconsistent data across nodes
Slow DB Access	Every request hits the database → high latency, DB overload under heavy payment traffic
Sticky Sessions	Load balancers forced to route a user to the same server → single point of failure
Coordination	Distributed locks for double-spend prevention require a coordination layer
Task Queues	Payment jobs (retry, settlement) need durable distributed queuing
Event Fanout	Payment events must be broadcast to all service instances simultaneously

1.3 Hazelcast Editions

Edition	Description
Hazelcast CE (Community)	Fully open-source under Apache 2.0. All core data structures, clustering, WAN replication basics. Sufficient for most use cases.
Hazelcast Enterprise	Adds security (TLS, LDAP, Kerberos), advanced WAN replication, Hot Restart Persistence, Blue-Green deployments, rolling upgrades.
Hazelcast Cloud	Managed Hazelcast as a service. Zero ops overhead, auto-scaling, built-in monitoring.

1.4 Hazelcast vs Alternatives

Technology	Type	Best For
Hazelcast	Distributed in-memory grid + compute	Best for: caching + session + distributed locking + events together
Redis	In-memory data store (external process)	Best for: simple caching, pub/sub; no native Java compute
Ehcache	Embedded local cache (or with Terracotta)	Best for: single-node local caching; no native cluster compute
Apache Ignite	Distributed in-memory + SQL	Best for: SQL queries on distributed data

Chapter 2: Hazelcast Architecture — How It Actually Works

2.1 The Cluster

A Hazelcast cluster is a group of Member nodes (JVM processes) that automatically discover each other, share data, and coordinate. Every member is equal — there is no master/slave hierarchy in the traditional sense (though Hazelcast does elect a 'master' member to manage cluster metadata, called the cluster coordinator or oldest member).

When you start your Spring Boot application with Hazelcast embedded, that JVM process becomes a member of the cluster. Start two instances — they form a 2-member cluster. Add a third — now you have 3. All three automatically know about each other and redistribute data.

💡 What is a Node vs a Member vs a Client?

These three words appear throughout Hazelcast docs and often cause confusion. Here is the precise meaning of each:

NODE -- A physical or virtual machine (or a Docker container / Kubernetes pod). One node can run one or more JVM processes. The word 'node' refers to the infrastructure.

MEMBER -- A JVM process that has joined the Hazelcast cluster. It holds a slice of the distributed data (partitions) and participates in cluster operations. When people say 'a Hazelcast member', they mean a running JVM that IS part of the cluster. In embedded mode, your Spring Boot JVM is a member. In standalone mode, each hazelcast.jar process is a member.

CLIENT -- A JVM process that CONNECTS to the cluster but does NOT join it and does NOT hold any data. Clients send operations (get, put, poll) to members over TCP. In client-server mode, your Spring Boot app is a client, not a member. In practice: one node usually runs one member (or one client). The words are often used interchangeably in casual conversation, but understanding the distinction is essential when designing your topology.

2.2 Member Discovery — How Nodes Find Each Other

Hazelcast supports multiple discovery mechanisms. This is how members discover peers on startup:

Mechanism	How It Works
Multicast (default dev)	Members broadcast a UDP multicast packet on 224.2.2.3:54327. Any other Hazelcast member on the same network LAN responds. Simple but doesn't work across subnets or in cloud environments.
TCP/IP	You configure a fixed list of seed member IP addresses. Members try to connect to each seed. Works in any environment. Best for containers and VMs.

Mechanism	How It Works
AWS / Azure / GCP	Cloud-native plugins use cloud APIs (e.g., AWS EC2 describe-instances) to discover members by tags. Fully dynamic.
Kubernetes	Hazelcast k8s plugin uses the Kubernetes API to list pods by label selector and connects to them.
Eureka / Consul / Zookeeper	Pluggable discovery via service registries. Members register themselves and query for peers.

2.3 Data Partitioning — How Data Is Distributed

This is the core mechanism that makes Hazelcast scale. All distributed data structures (IMap, IList, IQueue, etc.) are transparently partitioned across all cluster members.

Step-by-step: What happens when you call `map.put("payment:TXN001", payment)`

- Hazelcast takes the key ("payment:TXN001") and computes a partition ID: `partitionId = hash(key) % 271`
- 271 is the total number of partitions (fixed constant — never changes regardless of node count)
- Each partition is assigned to exactly one member (the partition owner)
- The member that handles your `put()` call routes it to the correct partition owner
- The partition owner stores the data in its local heap memory
- Hazelcast creates backup copies on different members (default: 1 backup)

With 3 members, approximately 90 partitions live on each member. When you add a 4th member, Hazelcast automatically migrates partitions from existing members to the new one without downtime — this is called repartitioning.

 **Payment Domain Example**

You store 10,000 payment records in an IMap. With 3 servers, roughly 3,333 records live on each server's memory. When a client calls `map.get("TXN001")`, Hazelcast computes the partition of "TXN001", discovers that partition 47 is owned by Server 2, and routes the request there — all transparently in under 1ms.

2.4 Backup & High Availability

By default, Hazelcast keeps 1 synchronous backup of every partition on a different member. This means:

- If Server 2 dies: Server 1 or Server 3 already has the backup of Server 2's partitions
- Hazelcast promotes the backup to primary instantly (within seconds)
- Zero data loss for the default sync backup configuration
- You can configure async backups for higher write throughput at the cost of potential data loss on sudden failure

Type	Behavior
Sync Backup (default)	The put() call blocks until the backup is written on the backup member. Zero data loss. Slightly higher latency.
Async Backup	The put() returns immediately; backup written asynchronously. Higher throughput. Risk of data loss if primary dies before backup completes.
Backup Count	You can set backup count to 0 (no HA), 1 (default), 2, or up to N-1 members. More backups = more memory used.

2.5 Split-Brain Protection

A split-brain occurs when network partition causes cluster members to separate into two isolated groups, both thinking they are the primary cluster. Both groups may accept writes to the same keys — creating a conflict.

Hazelcast handles split-brain with a Quorum (minimum cluster size) configuration and a merge policy. When the network heals, Hazelcast runs a merge: by default it uses the PutIfAbsentMergePolicy (keeps the existing entry if both exist). For payment systems, you'd use a custom merge policy that resolves conflicts by timestamp or transaction state.

2.6 Embedded Mode vs Client-Server Mode

Mode	How It Works
Embedded Mode	Your Spring Boot JVM IS a cluster MEMBER. It holds data partitions in its own heap. There is no separate Hazelcast process — the cluster IS your app processes. Zero network hop for data access (data is in the same JVM). Downside: every time you redeploy or restart your app, that member leaves the cluster and its partitions must be redistributed to the remaining members.
Client-Server Mode	Hazelcast MEMBERS run as completely separate JVM processes (standalone Hazelcast nodes). Your Spring Boot app runs as a Hazelcast CLIENT — it joins no cluster, holds

Mode	How It Works
	no data, and has no partitions. It simply connects to the member nodes over TCP to send commands. The cluster and all its data stay alive even when your app restarts. This is the production-recommended topology.

 **Key rule to remember**

MEMBER = stores data, participates in the cluster, owns partitions.
CLIENT = sends commands to members, stores nothing, invisible to the cluster topology.
In Embedded mode: your app JVM = MEMBER
In Client-Server mode: your app JVM = CLIENT, standalone Hazelcast JVMs = MEMBERS

For this guide we use Embedded Mode for theory and initial examples, then show Client-Server configuration in the setup chapter.

Chapter 3: Setup — Spring Boot + Hazelcast from Zero

3.1 Maven Dependencies

Add the following to your pom.xml. We use hazelcast-spring which auto-configures Hazelcast as the Spring Cache provider.

```
<!-- pom.xml -->
<dependencies>

    <!-- Spring Boot Starter Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Cache Abstraction -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-cache</artifactId>
    </dependency>

    <!-- Hazelcast Spring Integration (includes hazelcast core) -->
    <dependency>
        <groupId>com.hazelcast</groupId>
        <artifactId>hazelcast-spring</artifactId>
        <version>5.3.6</version>
    </dependency>

    <!-- Hazelcast Kubernetes Discovery (for k8s deployments) -->
    <dependency>
        <groupId>com.hazelcast</groupId>
        <artifactId>hazelcast-kubernetes</artifactId>
        <version>2.2.3</version>
    </dependency>

</dependencies>
```

3.2 Hazelcast Configuration — Java Config (Recommended)

Create a HazelcastConfig.java class. This is the main configuration class. Every setting is explained inline.

```
// HazelcastConfig.java
```

```
package com.payment.config;

import com.hazelcast.config.*;
import com.hazelcast.core.*;
import org.springframework.context.annotation.*;

@Configuration
public class HazelcastConfig {

    @Bean
    public Config hazelcastConfig() {
        Config config = new Config();

        // — Instance name (identifies this node in logs) —
        config.setInstanceName("payment-hazelcast-instance");

        // — Network Configuration —————
        NetworkConfig network = config.getNetworkConfig();
        network.setPort(5701); // default Hazelcast port
        network.setPortAutoIncrement(true); // if 5701 busy, try 5702...
        network.setPortCount(10); // try 10 ports max

        // — Join Config (TCP/IP for local/Docker) —————
        JoinConfig join = network.getJoin();
        join.getMulticastConfig().setEnabled(false); // disable multicast
        join.getTcpIpConfig()
            .setEnabled(true)
            .addMember("127.0.0.1"); // seed members

        // — IMap configs (one per map name) —————
        config.addMapConfig(paymentCacheConfig());
        config.addMapConfig(sessionCacheConfig());
        config.addMapConfig(idempotencyConfig());

        return config;
    }

    // Payment result cache: TTL 10 min, 1 backup
    private MapConfig paymentCacheConfig() {
        MapConfig mapConfig = new MapConfig("payment-cache");
        mapConfig.setTimeToLiveSeconds(600); // evict after 10 min
        mapConfig.setMaxIdleSeconds(300); // evict if idle 5 min
        mapConfig.setBackupCount(1); // 1 sync backup
        mapConfig.setAsyncBackupCount(0);
        mapConfig.setEvictionConfig(
            new EvictionConfig()
                .setEvictionPolicy(EvictionPolicy.LRU) // Least Recently Used
                .setMaxSizePolicy(MaxSizePolicy.PER_NODE)
                .setSize(50_000) // max 50k entries/node
        );
        // Near Cache: reduce remote lookups from same JVM
    }
}
```

```
        NearCacheConfig nearCache = new NearCacheConfig("payment-cache");
        nearCache.setTimeToLiveSeconds(60);
        nearCache.setMaxIdleSeconds(30);
        nearCache.setInvalidateOnChange(true);    // invalidate on remote put()
        mapConfig.setNearCacheConfig(nearCache);
        return mapConfig;
    }

    // Session store: TTL 30 min
    private MapConfig sessionCacheConfig() {
        MapConfig mapConfig = new MapConfig("spring:session:sessions");
        mapConfig.setTimeToLiveSeconds(1800);
        mapConfig.setBackupCount(1);
        return mapConfig;
    }

    // Idempotency keys: TTL 24 hours (prevent duplicate payments)
    private MapConfig idempotencyConfig() {
        MapConfig mapConfig = new MapConfig("idempotency-keys");
        mapConfig.setTimeToLiveSeconds(86_400);
        mapConfig.setBackupCount(2); // more backups = more safety for idempotency
        return mapConfig;
    }

    @Bean
    public HazelcastInstance hazelcastInstance(Config config) {
        return Hazelcast.newHazelcastInstance(config);
    }
}
```

3.3 YAML Configuration (Alternative)

If you prefer YAML, place `hazelcast.yaml` in `src/main/resources/`:

```
# hazelcast.yaml
hazelcast:
  instance-name: payment-hazelcast-instance
  network:
    port:
      auto-increment: true
      port-count: 10
      port: 5701
    join:
      multicast:
        enabled: false
      tcp-ip:
        enabled: true
        member-list:
          - 127.0.0.1
  map:
    payment-cache:
```

```
time-to-live-seconds: 600
max-idle-seconds: 300
backup-count: 1
eviction:
  eviction-policy: LRU
  max-size-policy: PER_NODE
  size: 50000
near-cache:
  time-to-live-seconds: 60
  invalidate-on-change: true
idempotency-keys:
  time-to-live-seconds: 86400
  backup-count: 2
```

3.4 application.properties Settings

```
# application.properties

# Enable Spring caching
spring.cache.type=hazelcast

# Point to your hazelcast.yaml (if using YAML config)
spring.hazelcast.config=classpath:hazelcast.yaml

# For Spring Session with Hazelcast
spring.session.store-type=hazelcast
spring.session.hazelcast.map-name=spring:session:sessions
spring.session.timeout=30m
```

3.5 Enable Spring Cache in Main Class

```
@SpringBootApplication
@EnableCaching          // ← required to activate @Cacheable annotations
public class PaymentServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(PaymentServiceApplication.class, args);
    }
}
```

3.6 Client-Server Mode — Running Standalone Hazelcast

In Client-Server mode, the Hazelcast cluster runs as completely separate JVM processes from your Spring Boot application. These separate JVMs are the MEMBERS — they own all the data, partitions, and backups. Your Spring Boot application is a pure CLIENT — it holds no data, owns no partitions, and is completely invisible to the cluster topology.

This is the production-recommended architecture: your app can be restarted or redeployed without losing any cached data, the cluster scales independently from the app, and you can run dedicated Hazelcast member nodes with tuned JVM settings (large heap, GC-optimized) completely separate from your app nodes.

💡 Why we run 2 member nodes (not 1)

A single member node is technically a valid cluster -- your client app will connect and work fine. However:

1. BACKUPS: Hazelcast replicates each partition to a different member (backup-count=1 by default). With only 1 member, there is nowhere to send the backup -- so all backups are silently skipped. If that single member dies, ALL data is lost instantly.

2. HIGH AVAILABILITY: With 2+ members, if one member goes down, the other already holds the backup partitions and promotes them to primary within seconds. Zero data loss.

3. SINGLE POINT OF FAILURE: A 1-member cluster means your entire cache layer dies the moment that one JVM crashes, is restarted, or is upgraded.

RULE: Use 1 member only for local development. Use minimum 2 members for any shared/staging/production environment. 3+ members is the production standard for zero-downtime rolling upgrades.

💡 Topology summary: who is a member and who is a client

In the setup below:

hz-node1 → Hazelcast MEMBER (stores data, owns partitions, participates in cluster)

hz-node2 → Hazelcast MEMBER (stores backup partitions, provides HA)

payment-service → Hazelcast CLIENT (connects to the cluster, sends commands, holds zero data) You can run 1, 2, 3, or 100 payment-service instances -- each is a separate client. Adding more clients does NOT change the cluster topology or redistribute any data. Only adding more member nodes changes the cluster.

Option A: Run Hazelcast as a standalone JAR

```
# Download Hazelcast standalone distribution
```

```
wget https://repo1.maven.org/maven2/com/hazelcast/hazelcast/5.3.6/hazelcast-5.3.6.jar
```

```
# Start member (node 1) on port 5701
```

```
java -jar hazelcast-5.3.6.jar
```

```
# Start member (node 2) – auto-discovers node 1
```

```
java -jar hazelcast-5.3.6.jar
```

```
# You now have a 2-member Hazelcast cluster running independently.
```

```
# Your Spring Boot app will connect as a CLIENT – not a member.
```

Option B: Run Hazelcast via Docker (Recommended)

```
# Run node 1
docker run -d --name hz-node1 -p 5701:5701 \
  -e HZ_CLUSTERNAME=payment-cluster \
  hazelcast/hazelcast:5.3.6

# Run node 2 (linked to node 1)
docker run -d --name hz-node2 -p 5702:5701 \
  -e HZ_CLUSTERNAME=payment-cluster \
  --link hz-node1 hazelcast/hazelcast:5.3.6

# Confirm 2-member cluster in logs:
docker logs hz-node1 | grep 'Members'
# Members {size:2, ver:2} [
#   Member [172.17.0.2]:5701 - this
#   Member [172.17.0.3]:5701
# ]
```

Server-side hazelcast-server.yaml (on the standalone cluster, NOT the Spring app):

```
# hazelcast-server.yaml
hazelcast:
  cluster-name: payment-cluster # client MUST use same name
  network:
    port:
      port: 5701
      auto-increment: true
    join:
      multicast:
        enabled: false
      tcp-ip:
        enabled: true
        member-list:
          - 192.168.1.10
          - 192.168.1.11
          - 192.168.1.12
  map:
    payment-cache:
      time-to-live-seconds: 600
      backup-count: 1
    idempotency-keys:
```

```
time-to-live-seconds: 86400
backup-count: 2
```

3.7 Spring Boot as Hazelcast Client (ClientConfig)

When your app runs in Client-Server mode, you replace the server-side Config bean with a ClientConfig bean. The app no longer holds data — it purely sends and receives operations to/from the cluster. The HazelcastInstance you inject in your services is now a client instance, but the API (IMap, IQueue, ITopic etc.) is identical to embedded mode.

```
// HazelcastClientConfig.java
// Use this INSTEAD of the embedded HazelcastConfig
package com.payment.config;

import com.hazelcast.client.HazelcastClient;
import com.hazelcast.client.config.*;
import com.hazelcast.core.HazelcastInstance;
import org.springframework.context.annotation.*;

@Configuration
@Profile("client-server") // -Dspring.profiles.active=client-server
public class HazelcastClientConfig {

    @Bean
    public ClientConfig hazelcastClientConfig() {
        ClientConfig clientConfig = new ClientConfig();

        // Must match cluster-name on the server
        clientConfig.setClusterName("payment-cluster");

        // List ALL server member addresses
        ClientNetworkConfig network = clientConfig.getNetworkConfig();
        network.addAddress("192.168.1.10:5701");
        network.addAddress("192.168.1.11:5701");
        network.addAddress("192.168.1.12:5701");
        network.setConnectionTimeout(5000); // 5s per attempt

        // Retry connecting for 120s before giving up
        clientConfig.getConnectionStrategyConfig()
            .getConnectionRetryConfig()
            .setClusterConnectTimeoutMillis(120_000);

        // Client-side Near Cache (local JVM cache – no data stored here)
```



```

NearCacheConfig nearCache = new NearCacheConfig("payment-cache");
nearCache.setTimeToLiveSeconds(60);
nearCache.setInvalidateOnChange(true); // server pushes invalidations
clientConfig.addNearCacheConfig(nearCache);

// Labels (visible in Management Center)
clientConfig.addLabel("payment-service");

// Serialization – must match the server factory
clientConfig.getSerializationConfig()
    .addDataSerializableFactory(
        PaymentSerializationFactory.FACTORY_ID,
        new PaymentSerializationFactory()
    );

return clientConfig;
}

// Returns a CLIENT HazelcastInstance (not a member node)
@Bean
public HazelcastInstance hazelcastInstance(ClientConfig cfg) {
    return HazelcastClient.newHazelcastClient(cfg);
}
}

```

💡 Key difference: Hazelcast vs HazelcastClient

Embedded: Hazelcast.newHazelcastInstance(config) creates a MEMBER node.
Client-Server: HazelcastClient.newHazelcastClient(clientConfig) creates a CLIENT. The returned HazelcastInstance interface is identical for both -- so ALL your service code (IMap, IQueue, ITopic, locks) works unchanged. Only the config class and constructor call differ.

Spring Profiles: switch between Embedded and Client-Server with zero code change:

```

// Two config classes, two profiles – your service code never changes

@Configuration @Profile("embedded")
public class HazelcastEmbeddedConfig {
    @Bean public Config hazelcastConfig() { /* section 3.2 */ }
    @Bean public HazelcastInstance hazelcastInstance(Config c) {
        return Hazelcast.newHazelcastInstance(c);
    }
}

```

```
}

@Configuration @Profile("client-server")
public class HazelcastClientConfig {
    @Bean public ClientConfig hazelcastClientConfig() { /* above */ }
    @Bean public HazelcastInstance hazelcastInstance(ClientConfig c) {
        return HazelcastClient.newHazelcastClient(c);
    }
}

# Local dev – embedded mode:
java -jar payment.jar --spring.profiles.active=embedded

# Production – client-server mode:
java -jar payment.jar --spring.profiles.active=client-server
```

hazelcast-client.yaml — YAML alternative for client config:

```
# src/main/resources/hazelcast-client.yaml
hazelcast-client:
  cluster-name: payment-cluster
  network:
    cluster-members:
      - 192.168.1.10:5701
      - 192.168.1.11:5701
      - 192.168.1.12:5701
    connection-timeout: 5000
  connection-strategy:
    connection-retry:
      cluster-connect-timeout-millis: 120000
  near-cache:
    payment-cache:
      time-to-live-seconds: 60
      invalidate-on-change: true
      eviction:
        eviction-policy: LRU
        max-size-policy: ENTRY_COUNT
        size: 10000

# application-client-server.properties
spring.cache.type=hazelcast
spring.hazelcast.config=classpath:hazelcast-client.yaml
spring.session.store-type=hazelcast
```

```
spring.session.hazelcast.map-name=spring:session:sessions
```

3.8 Docker Compose — Full Local Cluster Setup

The most practical way to develop and test Client-Server mode locally is with Docker Compose. The following file sets up a 2-node Hazelcast cluster + Management Center + your Spring Boot payment service as a client, all in one command.

```
# docker-compose.yml
version: '3.8'

services:

  hazelcast-node1:
    image: hazelcast/hazelcast:5.3.6
    container_name: hz-node1
    ports:
      - '5701:5701'
    environment:
      HZ_CLUSTERNAME: payment-cluster
      HZ_NETWORK_JOIN_TCPIP_ENABLED: 'true'
      HZ_NETWORK_JOIN_MULTICAST_ENABLED: 'false'
      HZ_NETWORK_JOIN_TCPIP_MEMBERLIST: hz-node1,hz-node2
      JAVA_OPTS: -Xms512m -Xmx1g -XX:+UseG1GC
    networks: [payment-net]
    healthcheck:
      test: [CMD, curl, -f, http://localhost:5701/hazelcast/health]
      interval: 10s
      timeout: 5s
      retries: 5

  hazelcast-node2:
    image: hazelcast/hazelcast:5.3.6
    container_name: hz-node2
    ports:
      - '5702:5701'
    environment:
      HZ_CLUSTERNAME: payment-cluster
      HZ_NETWORK_JOIN_TCPIP_ENABLED: 'true'
      HZ_NETWORK_JOIN_MULTICAST_ENABLED: 'false'
      HZ_NETWORK_JOIN_TCPIP_MEMBERLIST: hz-node1,hz-node2
    networks: [payment-net]
    depends_on:
```

```
    hazelcast-node1:
      condition: service_healthy

management-center:
  image: hazelcast/management-center:5.3.3
  container_name: hz-mgmt
  ports:
    - '8080:8080'
  environment:
    MC_DEFAULT_CLUSTER_MEMBERS: hz-node1:5701,hz-node2:5701
    MC_DEFAULT_CLUSTER_NAME: payment-cluster
    MC_ADMIN_USER: admin
    MC_ADMIN_PASSWORD: admin123
  networks: [payment-net]
  depends_on: [hazelcast-node1, hazelcast-node2]

payment-service:
  build: .
  container_name: payment-svc
  ports:
    - '8081:8080'
  environment:
    SPRING_PROFILES_ACTIVE: client-server
    HAZELCAST_CLIENT_ADDRESSES: hz-node1:5701,hz-node2:5701
  networks: [payment-net]
  depends_on:
    hazelcast-node1:
      condition: service_healthy

networks:
  payment-net:
    driver: bridge

# Start the full stack:
docker-compose up -d

# Watch cluster form (look for Members {size:2})
docker logs hz-node1 -f

# Open Management Center: http://localhost:8080 (admin / admin123)

# Test your payment service:
curl -X POST http://localhost:8081/payment \
  -H 'Content-Type: application/json' \
```

```
-d '{"txnId":"TXN001","amount":5000,"merchantId":"M001"}'
```

```
# Scale to 3 payment service instances (all share same cluster):
```

```
docker-compose up -d --scale payment-service=3
```

```
# Stop everything:
```

```
docker-compose down
```

What you should see in Management Center

After docker-compose up, open <http://localhost:8080>. You will see: 2 Hazelcast members (hz-node1, hz-node2) in the cluster view. Your payment-service listed under Connected Clients. Live map panel showing payment-cache and idempotency-keys being populated as you call your API. Memory and CPU metrics per member. Real-time operation counts (put/get/remove per second).

Chapter 4: IMap — The Core Data Structure

4.1 What is IMap?

IMap<K, V> is Hazelcast's distributed implementation of `java.util.concurrent.ConcurrentMap`. It is the single most used data structure in Hazelcast. When you put a key-value pair into an IMap, Hazelcast automatically:

- Computes the partition ID for the key using MurmurHash3
- Routes the operation to the member that owns that partition
- Stores the entry in that member's heap memory
- Replicates to backup members synchronously (by default)
- Returns to the caller

IMap is thread-safe, cluster-safe, and supports TTL, event listeners, near caches, entry processors, and predicates (server-side filtering).

4.2 Getting an IMap Instance

```
// Inject HazelcastInstance anywhere in your Spring components
@Autowired
private HazelcastInstance hazelcastInstance;

// Get (or create) a distributed map named 'payment-cache'
IMap<String, Payment> paymentMap =
    hazelcastInstance.getMap("payment-cache");

// NOTE: calling getMap() multiple times with the same name
// returns the same distributed map – it's cluster-wide singleton
```

4.3 Core Methods — Complete Reference

Basic CRUD Operations

Method Signature	Behavior
<code>put(K key, V value)</code>	Stores key-value. Blocks until backup written. Returns old value or null.
<code>put(K, V, ttl, TimeUnit)</code>	Stores with per-entry TTL override (overrides map-level TTL).
<code>putIfAbsent(K key, V value)</code>	Only stores if key doesn't exist. Returns existing value if key present. Atomic. Critical for idempotency.
<code>get(K key)</code>	Retrieves value. Returns null if not found or expired. Deserializes from member's memory.
<code>getOrDefault(K key, V default)</code>	Returns default value if key not found. Avoids null checks.
<code>remove(K key)</code>	Removes and returns the value. Propagates to backups.
<code>remove(K key, V oldValue)</code>	Conditional remove. Only removes if current value equals oldValue. Atomic.
<code>replace(K key, V value)</code>	Replaces only if key exists. Returns old value.
<code>replace(K, V oldValue, V newVal)</code>	Conditional replace. Only replaces if current == oldValue. Atomic. Optimistic locking.
<code>containsKey(K key)</code>	Returns true if key exists (not expired). Triggers partition lookup.
<code>size()</code>	Returns total number of entries across all partitions in all members.
<code>isEmpty()</code>	Returns true if no entries exist cluster-wide.
<code>clear()</code>	Removes ALL entries from ALL members. Use carefully.

Async Operations (Non-Blocking)

Every major operation has an async variant returning `CompletionStage<V>`:

```
// Async put – does not block the calling thread
CompletionStage<Payment> future = paymentMap.putAsync("TXN001", payment);
future.thenAccept(oldPayment -> {
    log.info("Stored, old value was: {}", oldPayment);
});

// Async get – fire-and-forget style
paymentMap.getAsync("TXN001")
    .thenApply(p -> p.getAmount())
    .thenAccept(amt -> log.info("Amount: {}", amt));

// Async remove
paymentMap.removeAsync("TXN001");

// Use for high-throughput batch writes
List<CompletionStage<?>> futures = txnIds.stream()
    .map(id -> paymentMap.putAsync(id, paymentFor(id)))
    .collect(Collectors.toList());
// Wait for all to complete
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
```

putAll — Batch Write (High Performance)

```
// putAll is MORE efficient than N individual put() calls
// It groups entries by partition and sends one batch per partition owner
Map<String, Payment> batch = new HashMap<>();
batch.put("TXN001", payment1);
batch.put("TXN002", payment2);
batch.put("TXN003", payment3);
paymentMap.putAll(batch); // one network call per partition (not per entry)

// Similarly for bulk read:
Set<String> keys = Set.of("TXN001", "TXN002", "TXN003");
Map<String, Payment> results = paymentMap.getAll(keys);
```


Locking Methods

IMap provides distributed locking per key. This is critical for payment processing where double-spend prevention is required.

Method	Behavior
<code>lock(K key)</code>	Acquires a distributed lock on the key. Blocks until acquired. The lock is owned by the current Hazelcast member+thread combination.
<code>lock(K key, time, TimeUnit)</code>	Acquires lock with timeout. Throws if timeout expires without acquiring.
<code>tryLock(K key)</code>	Non-blocking. Returns true if lock acquired, false if already held by another thread.
<code>tryLock(K, time, TimeUnit)</code>	Tries to acquire with timeout. Returns boolean.
<code>unlock(K key)</code>	Releases the lock. Must be called in a finally block.
<code>forceUnlock(K key)</code>	Releases lock regardless of owner. Use for recovery only.
<code>isLocked(K key)</code>	Returns true if any thread currently holds the lock for this key.

```
// Payment processing with distributed lock – prevents double spend
@Service
public class PaymentProcessor {

    @Autowired HazelcastInstance hz;

    public PaymentResult processPayment(String txnId, Payment payment) {
        IMap<String, Payment> map = hz.getMap("payments");

        // Lock this specific transaction ID across all cluster members
        map.lock(txnId);
        try {
            // Is this a duplicate?
            if (map.containsKey(txnId)) {
                return PaymentResult.DUPLICATE;
            }
            // Process payment (call bank API, debit account, etc.)
            PaymentResult result = bankGateway.charge(payment);
            if (result.isSuccess()) {
                map.put(txnId, payment, 24, TimeUnit.HOURS);
            }
            return result;
        } finally {
            map.unlock(txnId); // ALWAYS release in finally block!
        }
    }
}
```

Entry Processors — Server-Side Execution

Instead of: get → modify locally → put back (3 network calls + deserialization/serialization), an `EntryProcessor` runs your logic **DIRECTLY** on the partition owner's JVM. This is a single network call with no serialization of the full value.

```
// EntryProcessor to update payment status without full get/put cycle
public class PaymentStatusUpdater
    implements EntryProcessor<String, Payment, Boolean> {

    private final String newStatus;
    private final String updatedBy;

    public PaymentStatusUpdater(String newStatus, String updatedBy) {
        this.newStatus = newStatus;
        this.updatedBy = updatedBy;
    }

    @Override
    public Boolean process(Map.Entry<String, Payment> entry) {
        Payment payment = entry.getValue();
        if (payment == null) return false;

        // This code runs ON the partition owner – no network hop for the data!
        payment.setStatus(newStatus);
        payment.setUpdatedBy(updatedBy);
        payment.setUpdatedAt(Instant.now());
        entry.setValue(payment); // persist back
        return true;
    }
}

// Usage – update a single entry:
Boolean updated = paymentMap.executeOnKey(
    "TXN001",
    new PaymentStatusUpdater("SETTLED", "batch-job")
);

// Usage – update ALL entries matching a predicate:
Predicate<String, Payment> pendingPredicate =
    Predicates.equal("status", "PENDING");
Map<String, Boolean> results = paymentMap.executeOnEntries(
    new PaymentStatusUpdater("FAILED", "timeout-job"),
    pendingPredicate
);
```

Predicates — Server-Side Filtering

Predicates let you query the IMap without retrieving all entries to the client. Filtering runs on the partition owners. For large maps (millions of entries) this is vastly more efficient.

```
import com.hazelcast.query.Predicates;

IMap<String, Payment> payments = hz.getMap("payments");

// Get all FAILED payments
Collection<Payment> failed = payments.values(
    Predicates.equal("status", "FAILED")
);

// Get payments over 10,000 rupees
Collection<Payment> large = payments.values(
    Predicates.greaterThan("amount", 10_000.0)
);

// Combine predicates: FAILED AND amount > 10000
Predicate<String, Payment> combined = Predicates.and(
    Predicates.equal("status", "FAILED"),
    Predicates.greaterThan("amount", 10_000.0)
);
Collection<Payment> highValueFailed = payments.values(combined);

// SQL-style predicate
Collection<Payment> viaSQL = payments.values(
    Predicates.sql("status = 'FAILED' AND amount > 10000")
);

// Pagination (important for large result sets!)
PagingPredicate<String, Payment> paging = Predicates.pagingPredicate(
    Predicates.equal("status", "PENDING"),
    Comparator.comparing(Map.Entry::getValue),
    100 // 100 entries per page
);
Collection<Payment> page1 = payments.values(paging);
paging.nextPage();
Collection<Payment> page2 = payments.values(paging);
```

Indexes — Speed Up Predicate Queries

By default, predicate queries scan all entries. Add indexes to make them fast (just like a database index):

```
// Add in your MapConfig
MapConfig mapConfig = new MapConfig("payments");

// Sorted index – good for range queries (greaterThan, lessThan, between)
mapConfig.addIndexConfig(
    new IndexConfig(IndexType.SORTED, "amount")
);

// Hash index – good for equality queries (equal, in)
mapConfig.addIndexConfig(
    new IndexConfig(IndexType.HASH, "status")
);

mapConfig.addIndexConfig(
    new IndexConfig(IndexType.HASH, "merchantId")
);

// Bitmap index – for low-cardinality boolean fields
mapConfig.addIndexConfig(
    new IndexConfig(IndexType.BITMAP, "isRefunded")
);
```

Map Event Listeners

Listen for changes to the map — additions, updates, removals, and evictions. Listeners are invoked asynchronously on the member where the event originates.

```
public class PaymentMapListener
    implements EntryAddedListener<String, Payment>,
               EntryUpdatedListener<String, Payment>,
               EntryRemovedListener<String, Payment>,
               EntryEvictedListener<String, Payment> {

    @Override
    public void entryAdded(EntryEvent<String, Payment> event) {
        Payment p = event.getValue();
        auditLog.record("PAYMENT_STORED", p.getTxnId(), event.getMember());
    }

    @Override
    public void entryUpdated(EntryEvent<String, Payment> event) {
        Payment old = event.getOldValue();
        Payment neu = event.getValue();
        if (!old.getStatus().equals(neu.getStatus())) {
```

```
        notificationService.statusChanged(neu.getTxnId(), neu.getStatus());
    }
}

@Override
public void entryRemoved(EntryEvent<String, Payment> event) {
    // payment expired or manually removed
}

@Override
public void entryEvicted(EntryEvent<String, Payment> event) {
    // TTL or max-size eviction – persist to DB if needed
    Payment evicted = event.getOldValue();
    if (evicted.getStatus().equals("PENDING")) {
        paymentRepository.save(evicted); // flush to DB before eviction
    }
}

// Register the listener
paymentMap.addEntryListener(new PaymentMapListener(), true);
// Second param 'true' = include values in events (slightly more overhead)
```

Chapter 5: Other Distributed Data Structures

5.1 IQueue — Distributed Queue

IQueue<E> is a distributed, blocking FIFO queue that extends java.util.concurrent.BlockingQueue. Backed by a single partition (not distributed), it supports producer-consumer patterns across cluster members.

Key Methods

Method	Behavior
<code>offer(E element)</code>	Adds element to tail. Returns true/false. Non-blocking.
<code>offer(E, timeout, TimeUnit)</code>	Adds with timeout. Blocks if queue is full up to timeout.
<code>put(E element)</code>	Adds element. Blocks indefinitely if queue full (capacity-bounded).
<code>poll()</code>	Retrieves and removes head. Returns null if empty.
<code>poll(timeout, TimeUnit)</code>	Retrieves and removes head. Blocks up to timeout if empty.
<code>take()</code>	Retrieves and removes head. Blocks indefinitely if empty. Classic consumer pattern.
<code>peek()</code>	Retrieves head WITHOUT removing it. Returns null if empty.
<code>size()</code>	Returns current number of elements.
<code>remainingCapacity()</code>	Returns how many more elements can be added before queue is full.
<code>drainTo(Collection<E> c)</code>	Moves all available elements to collection c. Atomic. High-performance batch drain.
<code>drainTo(Collection<E>, max)</code>	Moves at most max elements.

```
// Payment retry queue – producer side
@Service
public class PaymentRetryProducer {

    private final IQueue<FailedPayment> retryQueue;

    public PaymentRetryProducer(HazelcastInstance hz) {
        // Queue config: max 10,000 entries, backed by store on overflow
        this.retryQueue = hz.getQueue("payment-retry-queue");
    }

    public void scheduleRetry(Payment failed, int attemptNumber) {
        FailedPayment retry = FailedPayment.builder()
            .payment(failed)
```

```
        .attemptNumber(attemptNumber)
        .scheduledAt(Instant.now().plus(30, SECONDS))
        .build();

    boolean enqueued = retryQueue.offer(retry, 5, TimeUnit.SECONDS);
    if (!enqueued) {
        log.error("Retry queue full! Falling back to DB queue for: {}",
            failed.getTxnId());
    }
}

// Payment retry queue – consumer side
@Service
public class PaymentRetryConsumer {

    private final IQueue<FailedPayment> retryQueue;

    public PaymentRetryConsumer(HazelcastInstance hz) {
        this.retryQueue = hz.getQueue("payment-retry-queue");
    }

    @Scheduled(fixedDelay = 5000)
    public void processRetries() throws InterruptedException {
        // Drain up to 50 items per cycle
        List<FailedPayment> batch = new ArrayList<>();
        retryQueue.drainTo(batch, 50);

        for (FailedPayment failed : batch) {
            if (Instant.now().isAfter(failed.getScheduledAt())) {
                paymentProcessor.reprocess(failed);
            } else {
                retryQueue.offer(failed); // re-enqueue if not due yet
            }
        }
    }
}
```

5.2 ITopic — Distributed Pub/Sub

ITopic<M> is a distributed publish-subscribe messaging mechanism. When a message is published, ALL subscribed members receive it. This is perfect for broadcasting payment events across all service instances.

Method	Behavior
publish(M message)	Publishes message to all subscribers across the cluster. Fire and forget.

Method	Behavior
<code>addMessageListener(MessageListener)</code>	Subscribes to messages. Listener invoked in a Hazelcast thread. Returns registration ID.
<code>removeMessageListener(String regId)</code>	Unsubscribes by registration ID.
<code>getLocalTopicStats()</code>	Returns published message count and receive count for this member.

```
// Define your event message (must be Serializable)
public class PaymentEvent implements Serializable {
    private String txnId;
    private String eventType; // INITIATED, AUTHORIZED, SETTLED, FAILED, REFUNDED
    private BigDecimal amount;
    private Instant timestamp;
    // getters, setters, builder...
}
```

```
// Publisher – PaymentService
@Service
public class PaymentService {
    @Autowired HazelcastInstance hz;

    public void settlePayment(String txnId) {
        // ... settlement logic ...

        // Broadcast event to all cluster members
        ITopic<PaymentEvent> topic = hz.getTopic("payment-events");
        topic.publish(PaymentEvent.builder()
            .txnId(txnId)
            .eventType("SETTLED")
            .timestamp(Instant.now())
            .build());
    }
}
```

```
// Subscriber – notification, audit, analytics services (all get it)
@Service
public class PaymentEventSubscriber implements MessageListener<PaymentEvent> {

    @Autowired HazelcastInstance hz;

    @PostConstruct
    public void subscribe() {
        hz.getTopic("payment-events").addMessageListener(this);
    }

    @Override
    public void onMessage(Message<PaymentEvent> message) {
        PaymentEvent event = message.getMessageObject();
    }
}
```



```
        switch (event.getEventType()) {
            case "SETTLED" -> auditService.record(event);
            case "FAILED" -> alertService.sendAlert(event);
            case "REFUNDED" -> ledgerService.bookReversal(event);
        }
    }
}
```

5.3 ISet — Distributed Set

`ISet<E>` is a distributed set that prevents duplicate elements cluster-wide. Useful for maintaining lists of processed transaction IDs.

```
// Track processed idempotency keys
ISet<String> processedTxns = hz.getSet("processed-transactions");

// add() returns true if element was new, false if already present
boolean isNew = processedTxns.add("TXN001");
if (!isNew) {
    throw new DuplicateTransactionException("TXN001 already processed");
}

// Check without adding
boolean alreadyDone = processedTxns.contains("TXN001");

// Remove when refunded
processedTxns.remove("TXN001");

// Get all processed IDs
Set<String> allProcessed = new HashSet<>(processedTxns);
```

5.4 IList — Distributed List

`IList<E>` is a distributed, ordered list. Unlike `IMap` it is NOT partitioned — the entire list lives on one member (with backup). Use for small lists where order matters.

```
// Maintain audit trail for a payment
IList<AuditEntry> auditTrail = hz.getList("audit-TXN001");

// Add audit entries
auditTrail.add(new AuditEntry("CREATED", Instant.now(), "user-svc"));
auditTrail.add(new AuditEntry("VALIDATED", Instant.now(), "validation-svc"));
auditTrail.add(new AuditEntry("CHARGED", Instant.now(), "payment-svc"));

// Get entry by index
AuditEntry first = auditTrail.get(0);
```

```
// Iterate in insertion order
for (AuditEntry entry : auditTrail) {
    System.out.println(entry);
}

// Add listener for audit trail changes
auditTrail.addItemListener(new ItemListener<AuditEntry>() {
    @Override
    public void itemAdded(ItemEvent<AuditEntry> item) {
        externalAuditSystem.push(item.getItem());
    }
    @Override
    public void itemRemoved(ItemEvent<AuditEntry> item) { }
}, true);
```

5.5 IAtomicLong — Distributed Counter

A distributed atomic long counter. Critical for generating sequential payment reference numbers or rate limiting.

Method	Behavior
<code>get()</code>	Returns current value.
<code>set(long newValue)</code>	Sets value.
<code>incrementAndGet()</code>	Atomically increments and returns new value. Thread-safe across cluster.
<code>getAndIncrement()</code>	Returns current value then increments.
<code>decrementAndGet()</code>	Atomically decrements and returns new value.
<code>addAndGet(long delta)</code>	Adds delta and returns new value.
<code>compareAndSet(exp, upd)</code>	Atomically: if current == expected, set to update. Returns boolean.
<code>alter(IFunction<Long>)</code>	Apply a function atomically on the value.

```
// Generate sequential payment reference numbers
IAtomicLong paymentSeq = hz.getCPSubsystem()
    .getAtomicLong("payment-seq"); // use CP subsystem for strong consistency

public String generatePaymentRef() {
    long seq = paymentSeq.incrementAndGet();
    return String.format("PAY-%s-%010d",
        LocalDate.now().format(DateTimeFormatter.BASIC_ISO_DATE), seq);
}
// PAY-20240615-0000000001, PAY-20240615-0000000002, ...
```

```
// Rate limiting: max 1000 requests per minute per merchant
IAAtomicLong requestCount = hz.getCPSubsystem()
    .getAtomicLong("rate-limit-merchant-" + merchantId);

if (requestCount.incrementAndGet() > 1000) {
    throw new RateLimitExceededException(merchantId);
}
// Reset counter each minute via @Scheduled job
```

5.6 FencedLock — Distributed Lock (CP Subsystem)

For critical sections requiring strong consistency (CP = RAFT consensus), use `FencedLock` from the CP Subsystem. Unlike `IMap.lock()`, `FencedLock` is backed by Raft consensus and is correct even under network partitions.

```
// Get a CP-backed fenced lock
FencedLock lock = hz.getCPSubsystem().getLock("settlement-lock");

// Standard lock/unlock pattern
long fence = lock.lockAndGetFence();
try {
    // fence is a monotonically increasing token
    // you can pass it to external systems to validate the lock is still valid
    runSettlement(fence);
} finally {
    lock.unlock();
}

// Try lock with timeout (best for payment timeouts)
boolean acquired = lock.tryLock(2, TimeUnit.SECONDS);
if (!acquired) {
    throw new PaymentTimeoutException("Could not acquire settlement lock");
}
try {
    runSettlement(lock.getFence());
} finally {
    lock.unlock();
}
```

Chapter 6: Spring Cache Integration

6.1 How Spring Cache Works with Hazelcast

Spring Cache is an abstraction layer. You annotate your service methods with `@Cacheable`, `@CachePut`, `@CacheEvict` and Spring handles the cache operations. Hazelcast plugs in as the cache provider — Spring Cache calls Hazelcast `IMap` internally.

6.2 `@Cacheable` — Cache Method Results

When a `@Cacheable` method is called, Spring first checks if the result exists in cache. If yes, it returns the cached value without executing the method. If no, it executes the method, stores the result, and returns it.

```
@Service
public class PaymentQueryService {

    @Autowired
    private PaymentRepository paymentRepository;

    // Basic caching: cache name = 'payment-cache', key = txnId argument
    @Cacheable(value = "payment-cache", key = "#txnId")
    public Payment getPayment(String txnId) {
        log.info("Fetching from DB: {}", txnId); // only logs on cache miss
        return paymentRepository.findByTxnId(txnId)
            .orElseThrow(() -> new PaymentNotFoundException(txnId));
    }

    // Conditional caching: only cache if payment is SETTLED
    @Cacheable(
        value = "payment-cache",
        key = "#txnId",
        condition = "#result != null && #result.status == 'SETTLED'"
    )
    public Payment getSettledPayment(String txnId) {
        return paymentRepository.findByTxnId(txnId).orElse(null);
    }

    // Composite key: merchantId + date
    @Cacheable(value = "merchant-summary",
        key = "#merchantId + '-' + #date.toString()")
    public MerchantSummary getMerchantSummary(String merchantId,
        LocalDate date) {
        return analyticsRepository.computeSummary(merchantId, date);
    }

    // Use SpEL for object field as key
```

```
@Cacheable(value = "payment-cache", key = "#request.txnId")
public Payment getByRequest(PaymentRequest request) {
    return paymentRepository.findByTxnId(request.getTxnId())
        .orElseThrow(PaymentNotFoundException::new);
}
```

6.3 @CachePut — Always Execute and Update Cache

@CachePut ALWAYS executes the method AND updates the cache. Use it when you want both the DB write and cache update to happen.

```
@CachePut(value = "payment-cache", key = "#result.txnId")
public Payment createPayment(CreatePaymentRequest request) {
    Payment payment = Payment.from(request);
    Payment saved = paymentRepository.save(payment);
    // result is automatically cached after this method returns
    return saved;
}

@CachePut(value = "payment-cache", key = "#txnId")
public Payment updatePaymentStatus(String txnId, String newStatus) {
    Payment payment = paymentRepository.findByTxnId(txnId)
        .orElseThrow(PaymentNotFoundException::new);
    payment.setStatus(newStatus);
    return paymentRepository.save(payment);
    // Updated payment is now in cache AND DB
}
```

6.4 @CacheEvict — Remove from Cache

@CacheEvict removes entries from the cache. Use when data is deleted or when you need to force a fresh DB read.

```
// Evict single entry
@CacheEvict(value = "payment-cache", key = "#txnId")
public void deletePayment(String txnId) {
    paymentRepository.deleteByTxnId(txnId);
}

// Evict ALL entries in a cache (use cautiously!)
@CacheEvict(value = "merchant-summary", allEntries = true)
public void clearMerchantCache() {
    log.info("Clearing all merchant summary cache");
}

// Evict BEFORE method executes (default is AFTER)
@CacheEvict(value = "payment-cache", key = "#txnId", beforeInvocation = true)
public void forceRefresh(String txnId) {
    // cache cleared before this runs
    // next call to getPayment() will go to DB
}

// Multiple evictions in one method
@Caching(evict = {
    @CacheEvict(value = "payment-cache", key = "#txnId"),
    @CacheEvict(value = "merchant-summary", allEntries = true)
})
public void processRefund(String txnId) {
    refundRepository.save(new Refund(txnId));
}
```

Chapter 7: Spring Session with Hazelcast

7.1 The Problem Spring Session Solves

By default, HTTP sessions are stored in the JVM's local memory. When your payment gateway runs on 3 pods, a user who logs in via Pod 1 will get a 401 on Pod 2 — Pod 2 has no knowledge of that session. Traditionally you'd use sticky sessions (load balancer always routes same user to same pod) which creates a single point of failure.

Spring Session + Hazelcast stores sessions in the distributed Hazelcast cluster instead of local JVM memory. Any pod can serve any user's session. Sessions survive pod restarts.

7.2 Dependency & Configuration

```
<!-- pom.xml: Spring Session -->
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-hazelcast</artifactId>
  <version>3.2.1</version>
</dependency>

// SessionConfig.java
@Configuration
@EnableHazelcastHttpSession(
    maxInactiveIntervalInSeconds = 1800 // 30 min session timeout
)
public class SessionConfig {

    // Spring Session will automatically use the 'spring:session:sessions' map
    // No extra bean needed if HazelcastInstance is already defined
}

// Using session in a payment controller
@RestController
@RequestMapping("/payment")
public class PaymentController {

    @PostMapping("/init")
    public InitResponse initPayment(HttpSession session,
                                   @RequestBody PaymentRequest req) {
        // Store payment intent in distributed session
        session.setAttribute("paymentIntent", req);
        session.setAttribute("merchantId", req.getMerchantId());
        session.setAttribute("sessionStart", Instant.now().toString());

        // Any server can now serve this session!
        return new InitResponse(session.getId());
    }
}
```

```
@PostMapping("/confirm")
public ConfirmResponse confirmPayment(HttpSession session,
                                     @RequestBody ConfirmRequest req) {
    // Retrieve from Hazelcast – works on ANY cluster member
    PaymentRequest intent =
        (PaymentRequest) session.getAttribute("paymentIntent");
    if (intent == null) {
        throw new SessionExpiredException("Payment session not found");
    }
    return paymentService.confirm(intent, req.getOtp());
}
```


Chapter 8: Serialization — How Data Travels Between Nodes

8.1 Why Serialization Matters

When data crosses a node boundary (put on Node A, retrieved on Node B), Hazelcast must convert your Java object to bytes (serialize) and back to an object (deserialize). The format and efficiency of this process dramatically affects performance.

Type	Tradeoffs
Java Serializable	Java's built-in serialization. Easiest (just implement Serializable). But VERY slow and produces large byte arrays. Avoid for production.
Hazelcast DataSerializable	Hazelcast's own interface. Faster than Java serialization. You manually write and read fields. Good balance of speed and simplicity.
Hazelcast IdentifiedDataSerializable	Extends DataSerializable. Avoids class name in byte stream — uses integer IDs instead. Faster deserialization. Recommended for production.
Portable	Schema-aware serialization. Allows querying fields WITHOUT full deserialization. Best when using predicates on large objects.
Compact (Hazelcast 5+)	Newest format. Schema-evolved, no code generation required. Fastest option. Use for new projects.
Jackson / JSON	Via custom serializer. Human-readable but large payload. Good for debugging.

8.2 Implementing IdentifiedDataSerializable

```
// PaymentFactory.java – registers class IDs
public class PaymentSerializationFactory
    implements DataSerializableFactory {

    public static final int FACTORY_ID    = 1;
    public static final int PAYMENT_ID    = 1;
    public static final int AUDIT_ENTRY_ID = 2;

    @Override
    public IdentifiedDataSerializable create(int typeId) {
        return switch (typeId) {
            case PAYMENT_ID    -> new Payment();
            case AUDIT_ENTRY_ID -> new AuditEntry();
        };
    }
}
```

```
        default -> throw new IllegalArgumentException("Unknown type: " + typeId);
    };
}
}
```

// Payment.java – the serializable domain object

```
public class Payment implements IdentifiedDataSerializable {

    private String txnId;
    private BigDecimal amount;
    private String status;
    private String merchantId;
    private Instant createdAt;

    @Override
    public int getFactoryId() {
        return PaymentSerializationFactory.FACTORY_ID;
    }

    @Override
    public int getClassId() {
        return PaymentSerializationFactory.PAYMENT_ID;
    }

    @Override
    public void writeData(ObjectDataOutput out) throws IOException {
        out.writeString(txnId);
        out.writeString(amount.toString());
        out.writeString(status);
        out.writeString(merchantId);
        out.writeLong(createdAt.toEpochMilli());
    }

    @Override
    public void readData(ObjectDataInput in) throws IOException {
        this.txnId = in.readString();
        this.amount = new BigDecimal(in.readString());
        this.status = in.readString();
        this.merchantId = in.readString();
        this.createdAt = Instant.ofEpochMilli(in.readLong());
    }
}

// Register factory in HazelcastConfig:
config.getSerializationConfig()
    .addDataSerializableFactory(
        PaymentSerializationFactory.FACTORY_ID,
        new PaymentSerializationFactory()
    );
```

Chapter 9: Advanced Patterns — Payment Domain

9.1 Idempotency Key Pattern

A payment must never be charged twice even if the client retries the request. The idempotency key pattern uses `IMap.putIfAbsent()` to atomically claim a transaction ID before processing.

```
@Service
public class IdempotentPaymentService {

    private static final String IDEMPOTENCY_MAP = "idempotency-keys";

    @Autowired HazelcastInstance hz;
    @Autowired BankGateway bankGateway;
    @Autowired PaymentRepository paymentRepository;

    public PaymentResult processPayment(String idempotencyKey,
                                       PaymentRequest request) {

        IMap<String, PaymentResult> idempotencyMap =
            hz.getMap(IDEMPOTENCY_MAP);

        // STEP 1: Try to claim the idempotency key
        // putIfAbsent is ATOMIC – safe across all cluster members
        PaymentResult placeholder = PaymentResult.processing();
        PaymentResult existing =
            idempotencyMap.putIfAbsent(idempotencyKey, placeholder,
                                     24, TimeUnit.HOURS); // expires in 24h

        if (existing != null) {
            // Key already existed → this is a retry
            if (existing.isCompleted()) {
                return existing; // return original result
            }
            // Still processing → return 409 Conflict
            throw new DuplicatePaymentException(idempotencyKey);
        }

        // STEP 2: We own this key – process the payment
        PaymentResult result;
        try {
            result = bankGateway.charge(request);
            paymentRepository.save(result.toPayment());
        } catch (Exception e) {
            // On failure, remove the key so client can retry
            idempotencyMap.remove(idempotencyKey);
            throw new PaymentProcessingException(e);
        }
    }
}
```

```
// STEP 3: Store the final result (overwrites placeholder)
idempotencyMap.put(idempotencyKey, result, 24, TimeUnit.HOURS);

return result;
}
}
```

9.2 Near Cache — Reducing Network Hops

A Near Cache is a local in-memory cache on the client side (or within the same JVM). When you `get()` a value, Hazelcast first checks the Near Cache. If found — returns instantly with zero network call. If not found — fetches from the cluster and puts it in the Near Cache for next time.

```
// Near Cache config in HazelcastConfig.java
NearCacheConfig nearCache = new NearCacheConfig();
nearCache.setName("payment-cache");
nearCache.setTimeToLiveSeconds(60);           // near cache entry lives 60s
nearCache.setMaxIdleSeconds(30);
nearCache.setInvalidateOnChange(true);       // if remote entry changes, local copy is
invalidated
nearCache.setEvictionConfig(
    new EvictionConfig()
        .setEvictionPolicy(EvictionPolicy.LFU) // Least Frequently Used
        .setMaxSizePolicy(MaxSizePolicy.ENTRY_COUNT)
        .setSize(10_000) // max 10k entries in near cache
);

// IMPORTANT: Near Cache causes 'stale reads' window of up to
// nearCache.timeToLiveSeconds. For payment STATUS lookups this
// is acceptable. For balance checks, set TTL to 0 or disable near cache.

MapConfig mapConfig = new MapConfig("payment-cache");
mapConfig.setNearCacheConfig(nearCache);
```

9.3 MapStore — Write-Through / Write-Behind Persistence

MapStore connects your `IMap` to a persistent database. Hazelcast can be configured to write entries to the DB transparently (write-through) or in batches (write-behind). On startup, it can pre-load the map from the DB (`MapLoader`).

```
// PaymentMapStore.java
@Component
public class PaymentMapStore implements MapStore<String, Payment> {

    @Autowired PaymentRepository repository;

    // — Write-through: called on every put() —————
```

```
@Override
public void store(String key, Payment value) {
    repository.save(value);
}

// — Batch write (write-behind): flush buffer to DB ———
@Override
public void storeAll(Map<String, Payment> map) {
    repository.saveAll(map.values());
}

// — Called on cache miss: load from DB into map —————
@Override
public Payment load(String key) {
    return repository.findByTxnId(key).orElse(null);
}

// — Bulk load: preload all keys from DB on startup ———
@Override
public Map<String, Payment> loadAll(Collection<String> keys) {
    return repository.findAllByTxnIdIn(keys).stream()
        .collect(Collectors.toMap(Payment::getTxnId, p -> p));
}

// — Return all keys to preload on member startup —————
@Override
public Iterable<String> loadAllKeys() {
    // Only load last 24h of payments to avoid OOM
    return repository.findTxnIdsSince(Instant.now().minus(1, DAYS));
}

@Override public void delete(String key) {
    repository.deleteByTxnId(key);
}

@Override public void deleteAll(Collection<String> keys) {
    repository.deleteAllByTxnIdIn(keys);
}
}

// Register in MapConfig:
MapStoreConfig storeConfig = new MapStoreConfig();
storeConfig.setImplementation(paymentMapStore);
storeConfig.setEnabled(true);
// Write-through (sync): writeDelaySeconds = 0
storeConfig.setWriteDelaySeconds(0);
// OR Write-behind (async batch, 5s delay):
// storeConfig.setWriteDelaySeconds(5);
// storeConfig.setWriteBatchSize(100);
mapConfig.setMapStoreConfig(storeConfig);
```

9.4 Distributed Transaction Pattern

Hazelcast supports XA transactions for coordinating operations across multiple distributed maps atomically.

```
@Service
public class TransferService {

    @Autowired HazelcastInstance hz;

    // Transfer between two accounts atomically
    public void transfer(String fromAccountId, String toAccountId,
        BigDecimal amount) {

        TransactionContext context = hz.newTransactionContext(
            new TransactionOptions()
                .setTransactionType(TransactionType.TWO_PHASE)
                .setTimeout(10, TimeUnit.SECONDS)
        );

        context.beginTransaction();
        try {
            TransactionalMap<String, Account> accounts =
                context.getMap("accounts");

            Account from = accounts.getForUpdate(fromAccountId);
            Account to = accounts.getForUpdate(toAccountId);

            if (from.getBalance().compareTo(amount) < 0) {
                throw new InsufficientFundsException(fromAccountId);
            }

            from.debit(amount);
            to.credit(amount);

            accounts.put(fromAccountId, from);
            accounts.put(toAccountId, to);

            context.commitTransaction();
            log.info("Transfer complete: {} -> {} : {}",
                fromAccountId, toAccountId, amount);

        } catch (Exception e) {
            context.rollbackTransaction();
            throw new TransferFailedException(e);
        }
    }
}
```

Chapter 10: Management, Monitoring & Best Practices

10.1 Hazelcast Management Center

Hazelcast Management Center is a web UI for monitoring and managing your cluster. It shows live member status, map sizes, cache hit/miss rates, IQueue depths, CPU/memory usage, and more.

```
# Run Management Center via Docker
docker run -p 8080:8080 hazelcast/management-center:5.3.3

# Connect to it from your HazelcastConfig:
config.getManagementCenterConfig()
    .setScriptingEnabled(false)
    .setConsoleEnabled(false);

# Open browser: http://localhost:8080
# Default credentials: admin / admin
```

10.2 Metrics via Spring Boot Actuator + Micrometer

```
<!-- pom.xml: add Micrometer Hazelcast support -->
<dependency>
    <groupId>com.hazelcast</groupId>
    <artifactId>hazelcast-spring</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

# application.properties
management.endpoints.web.exposure.include=health,metrics,prometheus
management.metrics.export.prometheus.enabled=true

# Key metrics exposed:
# hazelcast.map.put.count{map=payment-cache}
# hazelcast.map.get.count{map=payment-cache}
# hazelcast.map.hits{map=payment-cache}
# hazelcast.map.owned.entry.count{map=payment-cache}
# hazelcast.topic.publish.operation.count{topic=payment-events}
```

10.3 Best Practices Summary

Practice	Why
Always use TTL	Set <code>timeToLiveSeconds</code> on every map. Unbounded maps cause OOM errors under sustained load.
Serialize efficiently	Use <code>IdentifiedDataSerializable</code> or <code>Compact</code> serialization. Never use <code>Java Serializable</code> in production.
Index before querying	Add <code>HASH</code> indexes for equality predicates and <code>SORTED</code> indexes for range predicates. Unindexed scans are $O(n)$.
Near Cache wisely	Enable near cache for read-heavy, write-rarely data (reference data, merchant configs). Disable for balances.
Use Entry Processors	For read-modify-write on large objects, use <code>EntryProcessor</code> to avoid double network + full serialization.
CP Subsystem for counters	Use <code>hz.getCPSubsystem().getAtomicLong()</code> for payment sequence numbers — not the <code>AP IAtomicLong</code> which can split-brain.
putIfAbsent for idempotency	The atomic <code>putIfAbsent</code> is the foundation of all idempotency key patterns in payment processing.
Write-behind with caution	Write-behind increases throughput but risks data loss if a member crashes before the write delay expires. For payment data, use write-through (<code>delay = 0</code>).
Partition-aware keys	Group related data in the same partition using <code>PartitioningStrategy</code> . E.g., all entries for <code>merchantId X</code> go to the same partition — reduces scatter-gather for queries.
Size your heap	Each member should have at least 2x the peak data size as heap (1x data + 1x backup + JVM overhead). Monitor GC pressure.

10.4 Common Pitfalls

💡 **Pitfall 1: ClassCastException on deserialization**

If two cluster members have different versions of your domain class (e.g., during rolling deploy), deserialization will fail. Use `IdentifiedDataSerializable` with versioned `readData/writeData` to handle schema evolution gracefully.

💡 **Pitfall 2: Unbounded Maps → OutOfMemoryError**

A map without TTL or max-size eviction will grow until the member runs out of heap. Always set `timeToLiveSeconds` and `eviction.size` on every `MapConfig`. Monitor `hazelcast.map.owned.entry.count` in Prometheus.

💡 **Pitfall 3: Forgetting to unlock()**

`IMap.lock()` must always be released in a finally block. A lock held by a crashed thread will be automatically released when the member loses its connection, but an unreleased lock in a still-running member will cause deadlock.

💡 **Pitfall 4: Using IAtomicLong for strong consistency**

The default `IAtomicLong` is AP (eventually consistent) and can diverge during split-brain. For payment sequence numbers, always use `hz.getCPSubsystem().getAtomicLong()` which uses RAFT consensus.

Chapter 11: Quick Reference Card

11.1 Data Structure Selection Guide

Use Case	Use	Payment Example
Store key-value data	IMap<K,V>	Payment records, session data, cache entries
Process items in order	IQueue<E>	Retry queue, settlement batch queue
Broadcast events to all members	ITopic<M>	Payment events, config refresh signals
Prevent duplicate entries	ISet<E>	Processed transaction ID registry
Maintain ordered list	IList<E>	Audit trail, payment history per transaction
Atomic counter / sequence	CP AtomicLong	Payment reference number generator
Distributed exclusive lock	CP FencedLock	Settlement job, ledger posting
Per-key optimistic lock	IMap.replace(K,old,new)	Status CAS updates
Atomic claim (idempotency)	IMap.putIfAbsent()	Idempotency key pattern
Server-side compute	EntryProcessor	Bulk status updates, conditional logic

11.2 Configuration Cheat Sheet

Property	Meaning
timeToLiveSeconds	Max age of any entry. Entry evicted after this regardless of access.
maxIdleSeconds	Max idle time. Entry evicted if not accessed for this duration.
backupCount	Number of synchronous backups (0-6). Default 1. Higher = more resilience, more memory.
asyncBackupCount	Async backup count. Higher write throughput, risk of data loss on crash.
evictionPolicy	LRU (default), LFU, RANDOM, NONE.
eviction.size	Max entries per partition. When exceeded, eviction policy applies.

Property	Meaning
<code>nearCacheConfig</code>	Client-side local cache to reduce remote <code>get()</code> calls.
<code>mapStoreConfig</code>	Read/write integration with persistent database.
<code>indexConfig</code>	SORTED (range) or HASH (equality) or BITMAP (boolean) index on an attribute.

— End of Document —

Hazelcast Deep Dive | Spring Boot | Payment Domain